INTRODUCTION :→

In recent years, online meeting platforms have become an essential part of communication in education, business, and remote collaboration. However, language differences often create barriers that reduce the effectiveness of communication. Participants may find it difficult to understand conversations, leading to misunderstandings and reduced productivity.

To overcome this challenge, the proposed system *BhashaBridge* is developed as a multilingual meeting platform that integrates real-time translation features into the communication process. The system enables users to interact through chat and audio while automatically translating content into their preferred language. By combining communication tools with intelligent translation services, the system aims to create a more inclusive and efficient meeting environment.

ABSTRACT :→

In the modern digital age, online communication platforms have become a fundamental part of daily life, especially in professional environments, education systems, and global collaboration. With the rise of remote work and virtual meetings, people from diverse linguistic backgrounds frequently interact with one another. However, one of the most significant challenges in such interactions is the language barrier, which often leads to misunderstandings, reduced efficiency, and limited participation. To address this issue, this project introduces *BhashaBridge*, an intelligent multilingual meeting platform designed to facilitate seamless and inclusive communication across different languages.

The system provides all the essential functionalities expected from a modern online meeting platform. Users can register, log in, create meetings, join existing sessions, and communicate through both chat and audio. These core features ensure that the platform supports smooth and effective interaction among participants. However, what makes *BhashaBridge* unique is its integration of advanced real-time translation capabilities that enhance communication beyond traditional meeting systems.

One of the key features of the system is message translation. When users send chat messages, the system can automatically translate them into multiple languages based on the preferences of other participants. This ensures that users can communicate freely in their own language while still being understood by others. In addition to text communication, the system also supports live audio interaction, which is further enhanced through intelligent processing.

The platform includes a live caption generation feature that converts spoken language into text in real time. This not only helps users follow conversations more easily but also improves accessibility for users who may have hearing difficulties. Furthermore, the generated captions can be translated into different languages, allowing participants to read and understand spoken content in their

preferred language. Another advanced feature is real-time audio translation, where spoken audio is directly translated and delivered to listeners, making communication even more natural and interactive.

An important aspect of the system design is that these translation features are optional. Users can choose whether they want to enable or disable translation services based on their needs. This flexibility ensures that the platform remains user-friendly and adaptable to different scenarios, whether users require translation support or prefer standard communication.

In addition to communication features, *BhashaBridge* also provides meeting analytics and reporting functionalities. At the end of each meeting, the system calculates important metrics such as the total number of participants and the duration of the meeting. Based on this data, a detailed report is generated, which helps users review the meeting outcomes and maintain records for future reference. These features are particularly useful in professional and organizational settings where tracking and documentation are important.

The system also incorporates a structured administrative model to ensure efficient management. There are two distinct types of administrators: Organization Admin and Platform Admin. The Organization Admin is responsible for managing users within a specific organization, including adding or removing users and assigning roles such as host or participant. On the other hand, the Platform Admin manages system-level operations, including monitoring system performance, managing services like translation and reporting, and ensuring the overall stability of the platform. This separation of responsibilities improves system organization and scalability, especially in a Software as a Service (SaaS) environment.

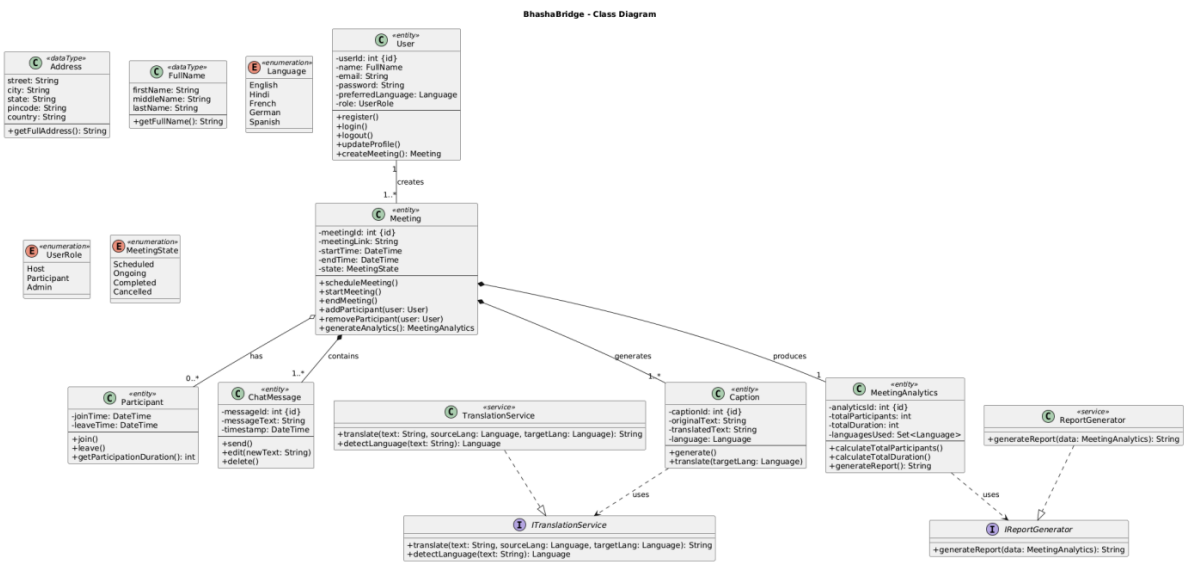
From a technical perspective, the system follows a modular and service-oriented architecture. The meeting system interacts with external services such as translation services and report generation services to perform specialized tasks. This design allows the system to remain scalable, flexible, and easy to maintain. New features and services can be integrated in the future without significantly affecting the existing system.

The primary objective of *BhashaBridge* is to create a communication platform where language is no longer a barrier. By combining traditional meeting functionalities with intelligent translation technologies, the system enhances accessibility, improves user experience, and promotes effective collaboration among users from different linguistic backgrounds. It aims to provide a practical and innovative solution that addresses real-world communication challenges.

In conclusion, *BhashaBridge* represents a significant step toward inclusive digital communication. By enabling real-time multilingual interaction and providing advanced meeting features, the system not only improves understanding but also fosters better collaboration and connectivity in a globally connected world. This project highlights the importance of integrating intelligent technologies into communication platforms to meet the evolving needs of users.

UML DIAGRAMS WITH EXPLANATION :→

CLASS DIAGRAM :→



- This diagram represents the **static structure of the system**, showing classes, attributes, methods, and relationships.

---

1. Core Classes

User

- Contains user details like id, name, email, password, preferred language.
- Performs actions like register, login, logout, update profile.
- Relationship: **User creates Meeting**

Meeting

- Contains meeting details like meetingId, link, startTime, endTime, state.
- Handles operations like start, end, add/remove participants.
- Central class of the system.

---

2. Supporting Classes

Participant

- Represents users inside a meeting.
- Stores join time and leave time.
- Relationship: **Meeting has multiple Participants (1 → many)**

ChatMessage

- Stores messages sent during meeting.
- Contains message text and timestamp.
- Relationship: **Meeting contains multiple messages**

Caption

- Represents text generated from audio.
- Contains original and translated text.
- Relationship: **Generated from Meeting / Audio**

MeetingAnalytics

- Stores analytics like:
  - total participants
  - total duration
  - languages used
- Relationship: **Produced by Meeting**

---

3. Service Classes

TranslationService

- Handles translation and language detection.
- Used by Caption and ChatMessage.
- Shows **dependency relationship**

ReportGenerator

- Generates reports from MeetingAnalytics.
- Shows **dependency on analytics**

---

4. Interfaces

- `ITranslationService` and `IReportGenerator`
- Provide abstraction for services.
- Shows **loose coupling and flexibility**

---

5. Enumerations

- **UserRole** → Host, Participant, Admin
- **MeetingState** → Scheduled, Ongoing, Completed, Cancelled
- **Language** → English, Hindi, etc.

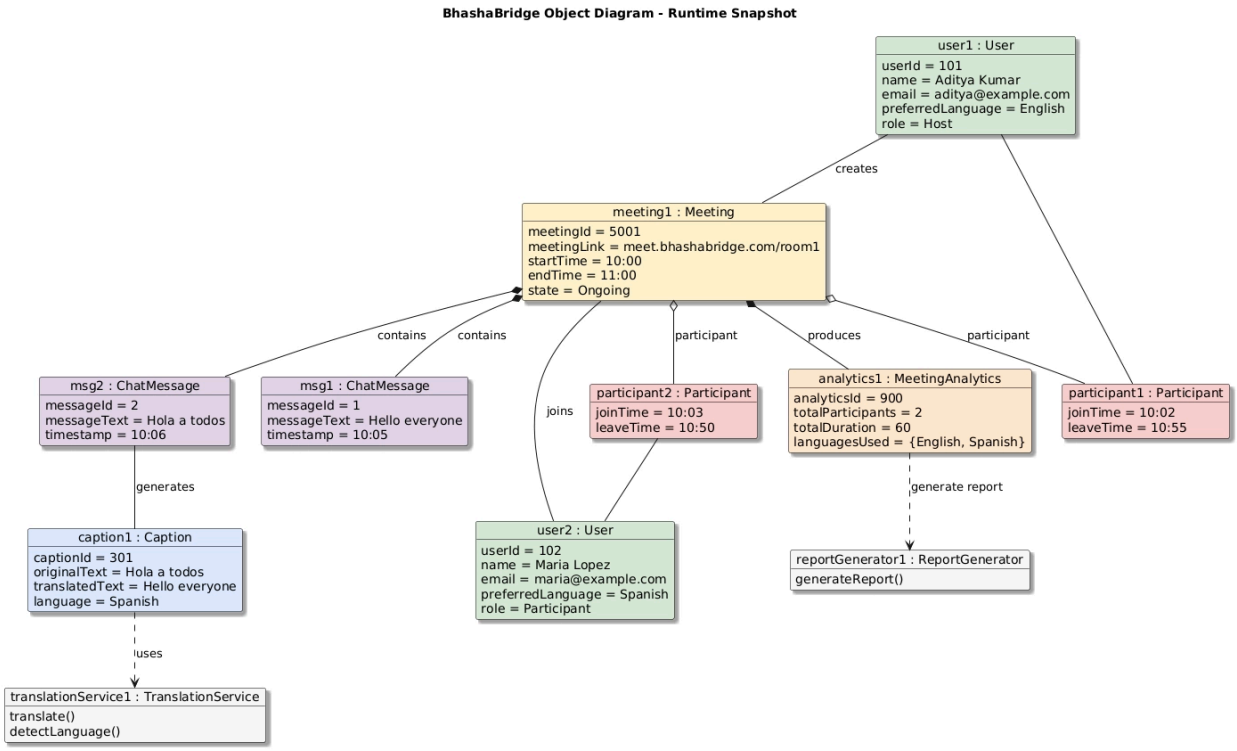
👉 Used for predefined values.

---

6. Key Relationships

- User → Meeting → (creates)
- Meeting → Participant → (has many)
- Meeting → ChatMessage → (contains)
- Meeting → Caption / Analytics → (generates)
- Services → used via dependency
- Interfaces → implemented by services
-

OBJECT DIAGRAM :→



- This object diagram represents the **runtime state of the system at a specific moment**, showing actual objects and their values.

---

1. User and Meeting Relationship

- `user1 : User` (Host) creates `meeting1 : Meeting`.
- This shows a **"creates" relationship**, meaning the host is responsible for starting the meeting.
- `user2 : User` participates in the meeting as a participant.

---

2. Meeting and Participants

- The meeting contains multiple participant objects:
  - `participant1`
  - `participant2`

- Each participant has attributes like join time and leave time.
- This represents a **one-to-many relationship** (one meeting → many participants).

---

3. Meeting and Chat Messages

- The meeting contains chat messages:
  - `msg1` (Hello everyone)
  - `msg2` (Hola a todos)
- This shows a **containment relationship**, where messages belong to the meeting.

---

4. Message and Caption Relationship

- `msg2` generates `caption1`.
- The caption includes:
  - original text
  - translated text
- This represents a **derivation relationship**, where captions are generated from messages.

---

5. Translation Service Usage

- The caption uses `translationService1`.
- This shows a **dependency relationship**, where the system relies on an external service for translation.

---

6. Meeting and Analytics

- The meeting produces `analytics1`.
- Analytics includes:

    - total participants
    - total duration
    - languages used
- This shows a **"produces" relationship**, meaning analytics is derived from meeting data.

---

7. Analytics and Report Generator

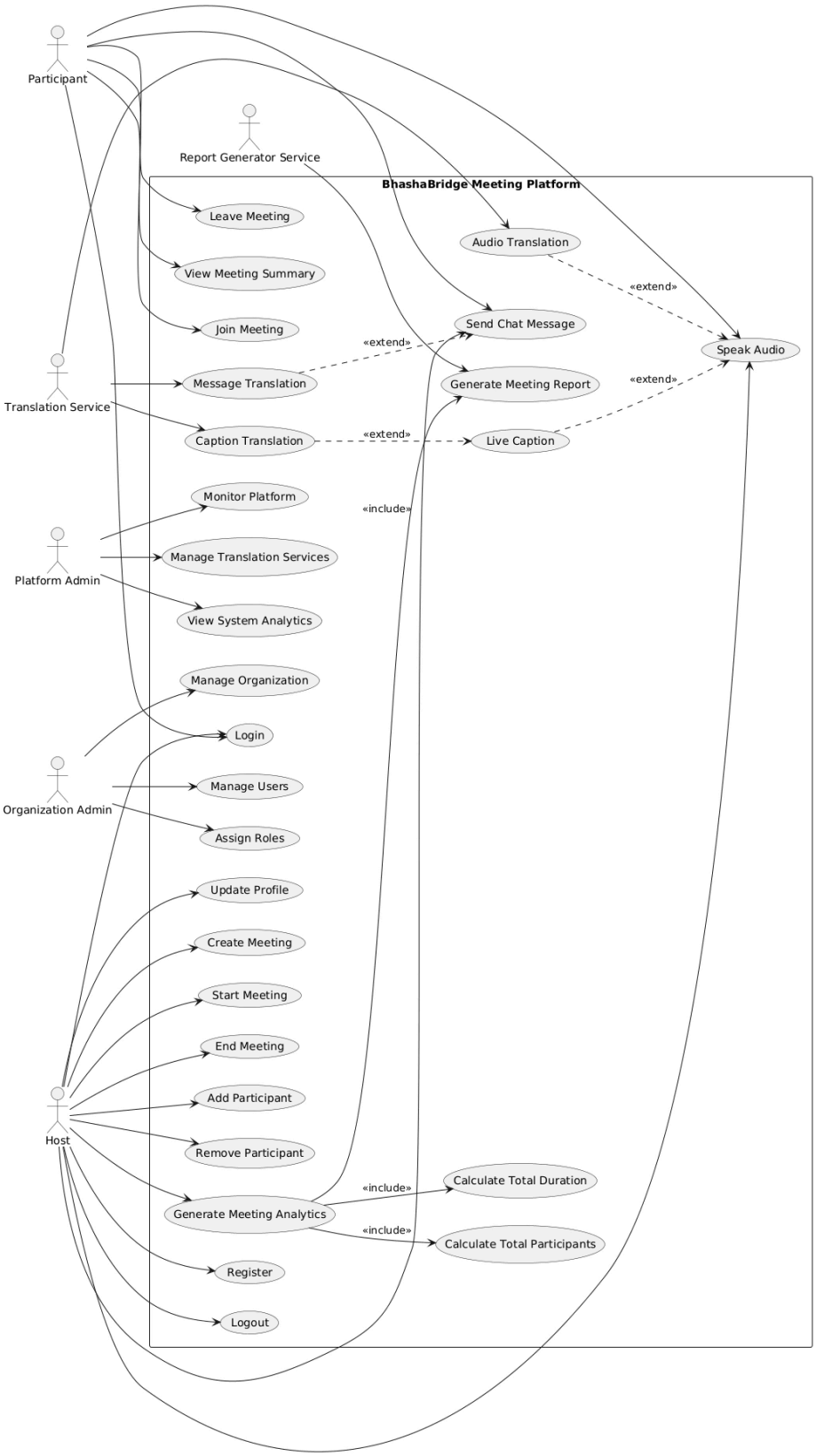
- `analytics1` is used by `reportGenerator1` to generate reports.
- This represents another **dependency relationship**, where reporting depends on analytics data.

---

8. Overall Structure

- The meeting acts as the **central object** connecting users, messages, participants, analytics, and services.
- External services like translation and report generation are used to enhance functionality.
-

USE CASE DIAGRAM :→


Participant
Report Generator Service
BhashaBridge Meeting Platform
Leave Meeting
Audio Translation
View Meeting Summary
«extend»
Join Meeting
Send Chat Message
Speak Audio
«extend»
Translation Service
Message Translation
Generate Meeting Report
«extend»
Caption Translation
«extend»
Live Caption
Monitor Platform
«include»
Platform Admin
Manage Translation Services
View System Analytics
Manage Organization
Login
Organization Admin
Manage Users
Assign Roles
Update Profile
Create Meeting
Start Meeting
End Meeting
Add Participant
Host
Remove Participant
«include»
Calculate Total Duration
Generate Meeting Analytics
«include»
Calculate Total Participants
Register
Logout

The Use Case Diagram represents the functional requirements of the BhashaBridge system and shows how different actors interact with the system. The primary actors include Host, Participant, Organization Admin, Platform Admin, Translation Service, and Report Generator Service.

The Host is responsible for creating and managing meetings, while the Participant joins meetings and interacts through chat and audio. Both actors can send messages and speak during meetings.

The diagram uses two important relationships:

- `<<extend>>` **relationship:**

  This represents optional functionality. For example:

  - *Message Translation* extends *Send Chat Message*
  - *Live Caption* extends *Speak Audio*
  - *Caption Translation* extends *Live Caption*
  - *Audio Translation* extends *Speak Audio*

  This means these features are executed **only when enabled**, making the system flexible.

- `<<include>>` **relationship:**

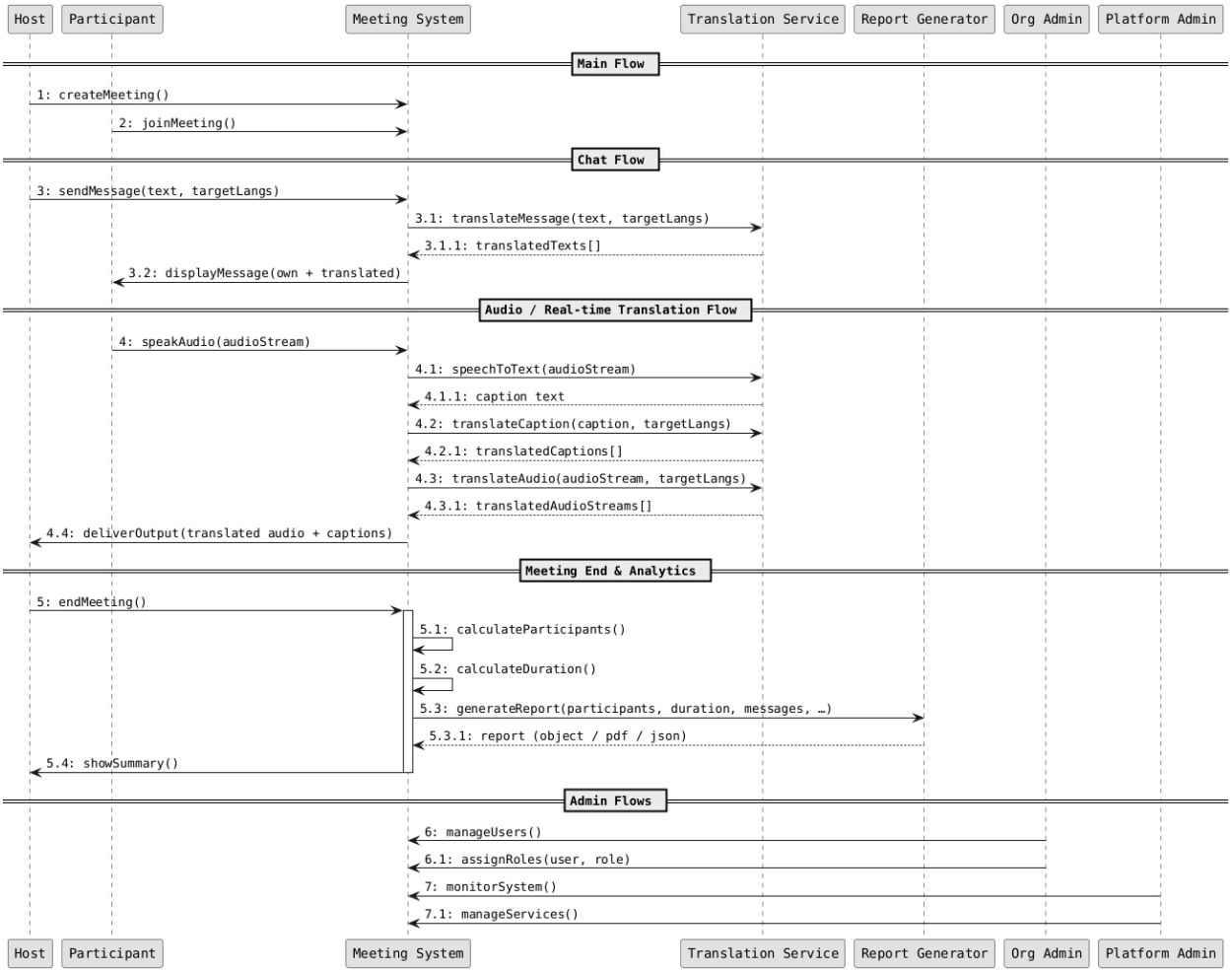
  This represents mandatory functionality. For example:

  - *Generate Meeting Analytics* includes:
    - Calculate Total Participants
    - Calculate Total Duration
    - Generate Meeting Report

  This means these steps are **always performed together**.

The Translation Service and Report Generator Service are external systems that support translation and reporting functionalities.

SEQUENCE DIAGRAM :→



🔹 Main Flow

- Host creates the meeting
- Participant joins the meeting
  👉 **User → Meeting System interaction**

🔹 Chat Flow

- User sends message
- Meeting System sends it to Translation Service (optional)
- Translated message is returned and displayed
  👉 **System depends on Translation Service**

🔹 Audio Flow

- User speaks audio
- System converts speech to text (caption)
- Caption and audio can be translated (optional)
- Final output is delivered
  👉 **Pipeline flow: Audio → Caption → Translation → Output**

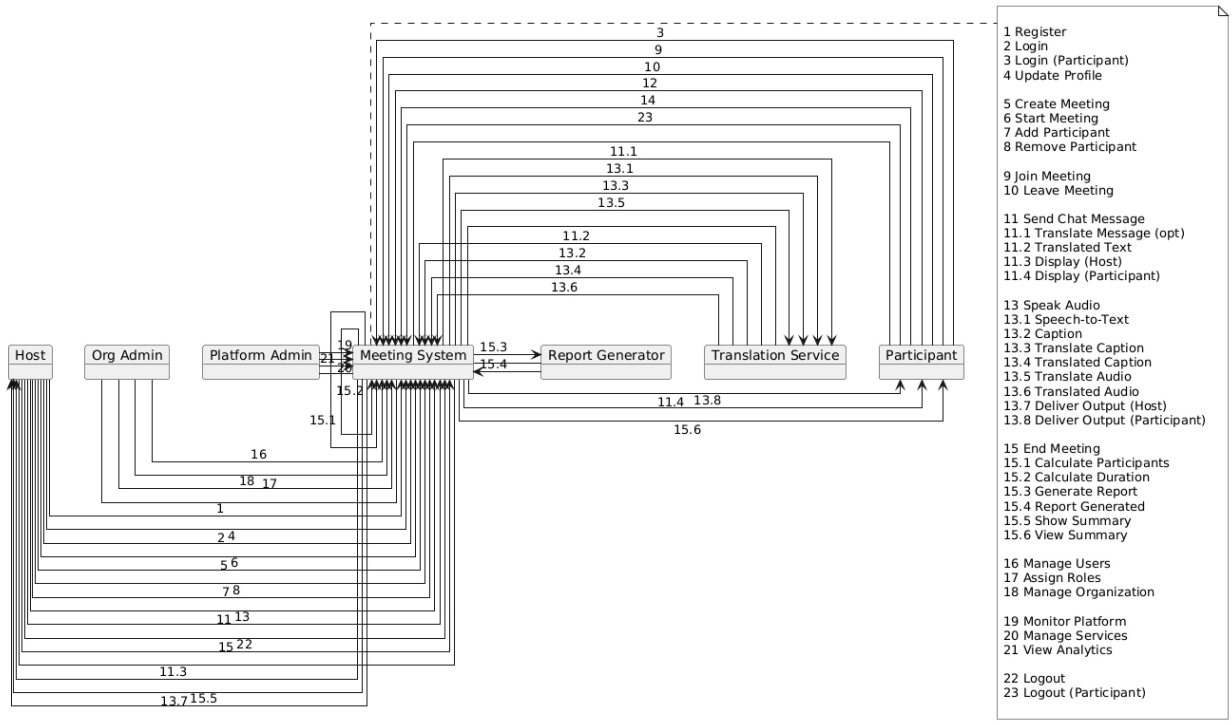
🔹 Meeting End & Report

- Host ends meeting
- System calculates participants & duration
- Report is generated via Report Service
  👉 **System → Report Generator dependency**

🔹 Admin Flow

- Organization Admin → manages users & roles
- Platform Admin → monitors system & services
  👉 **Role-based access control**

COLLABORATION DIAGRAM :→



This diagram shows how different components like **Host, Participant, Meeting System, Translation Service, Report Generator, and Admins** interact with each other.

🔹 Key Idea

👉 The **Meeting System is the central component**

All actors communicate through it.

🔹 Numbered Messages

- Numbers like **11, 11.1, 11.2** show **flow of execution**
- Example:
  - `11` → Send Message
  - `11.1` → Translate Message
  - `11.2` → Return translated text

👉 Shows **which action triggers next action**

🔹 Chat & Translation

- User sends message → Meeting System
- System → Translation Service (optional)
- Result → back to users
  👉 **Dependency on Translation Service**

🔹 Audio Flow

- User speaks → System
- System generates caption → translates → delivers output
  👉 **Step-by-step pipeline interaction**

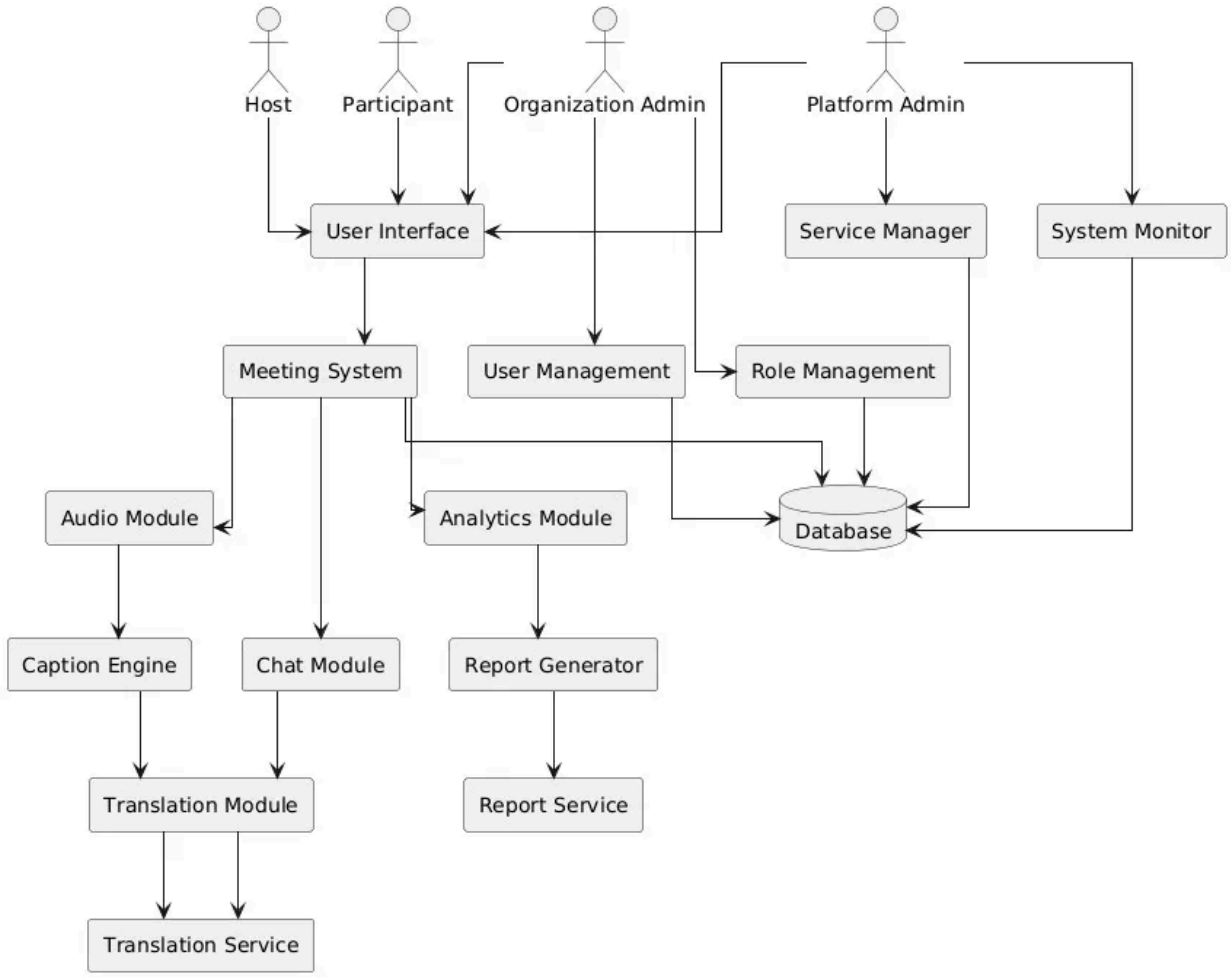
🔹 Report Generation

- Meeting ends → System calculates data
- System → Report Generator
  👉 **External service interaction**

🔹 Admin Control

- Org Admin → manages users
- Platform Admin → manages system
  👉 **Role-based control**

COMPONENT DIAGRAM :→



- This diagram represents the **high-level structure of the system** by showing different components and how they interact with each other.

---

1. User Interface Component

- Acts as the **entry point** for all users:
  - Host
  - Participant
  - Organization Admin
  - Platform Admin
- Sends user requests to the Meeting System.

---

2. Meeting System (Core Component)

- The **central component** of the system.
- Coordinates all other components such as:
  - Chat Module
  - Audio Module
  - Analytics Module
- Controls the overall meeting flow.

---

3. Communication Components

Chat Module

- Handles sending and receiving messages.
- Connected to Translation Module.

Audio Module

- Handles voice input.
- Sends audio to Caption Engine.

Caption Engine

- Converts speech into text (speech-to-text).

Translation Module

- Translates chat messages and captions.
- Uses Translation Service.

Translation Service

- External component providing translation functionality.
- Represents a **provided service**.

---

4. Analytics and Reporting Components

Analytics Module

- Processes meeting data like participants and duration.

Report Generator

- Generates reports from analytics data.

Report Service

- External service used for report generation.

---

5. Management Components

User Management

- Handles user-related operations.

Role Management

- Assigns roles to users.

---

6. Admin Components

Organization Admin

- Uses User Management and Role Management.

Platform Admin

- Uses:
  - Service Manager (manage services)
  - System Monitor (monitor system)

---

7. Database Component

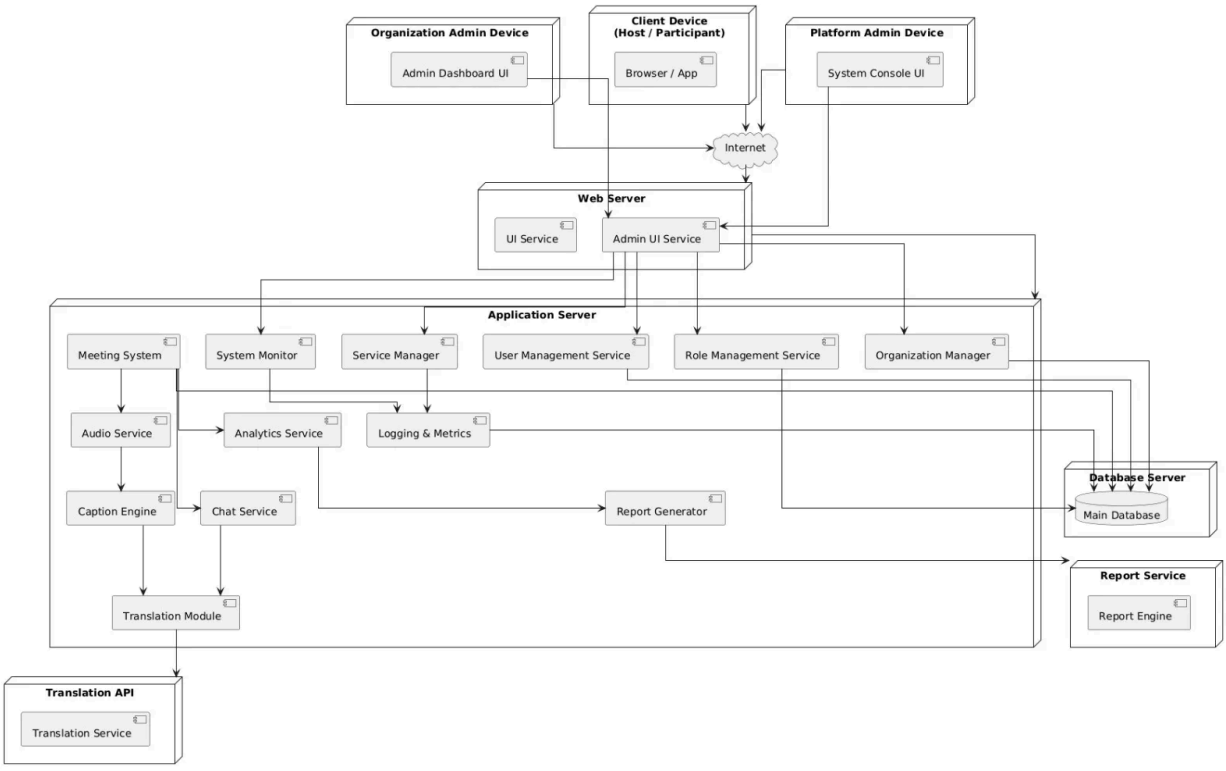
- Central storage used by multiple components.

- Stores user, meeting, and analytics data.
- Represents a **shared resource**.

KEY RELATIONSHIPS

- **Dependency**:
  Components depend on services (Translation Service, Report Service)
- **Association**:
  Components interact with each other (Meeting System ↔ Modules)
- **Central Control**:
  Meeting System connects and coordinates all components

DEPLOYMENT DIAGRAM :→



- This diagram shows the **physical deployment of the system**, including hardware nodes, servers, and how components are distributed.

---

1. Client Devices

- Includes:
  - Host / Participant device (Browser/App)
  - Organization Admin device (Admin Dashboard UI)
  - Platform Admin device (System Console UI)
- These devices interact with the system through the **Internet**.

---

2. Web Server

- Handles user requests from client devices.
- Contains:
  - UI Service
  - Admin UI Service
- Acts as a **bridge between client and backend system**.

---

3. Application Server

- Core processing unit of the system.
- Contains major services:
  - Meeting System
  - Audio Service, Chat Service
  - Caption Engine, Translation Module
  - User Management, Role Management
  - Analytics Service, Report Generator
  - System Monitor, Service Manager

- This shows a **service-based architecture** where different modules handle specific tasks.

---

4. Translation API (External Service)

- Used for language translation.
- Connected to Translation Module.
- Represents an **external dependency**.

---

5. Database Server

- Stores all system data:
  - users
  - meetings
  - analytics
- Shared by multiple services.
- Represents **central data storage**.

---

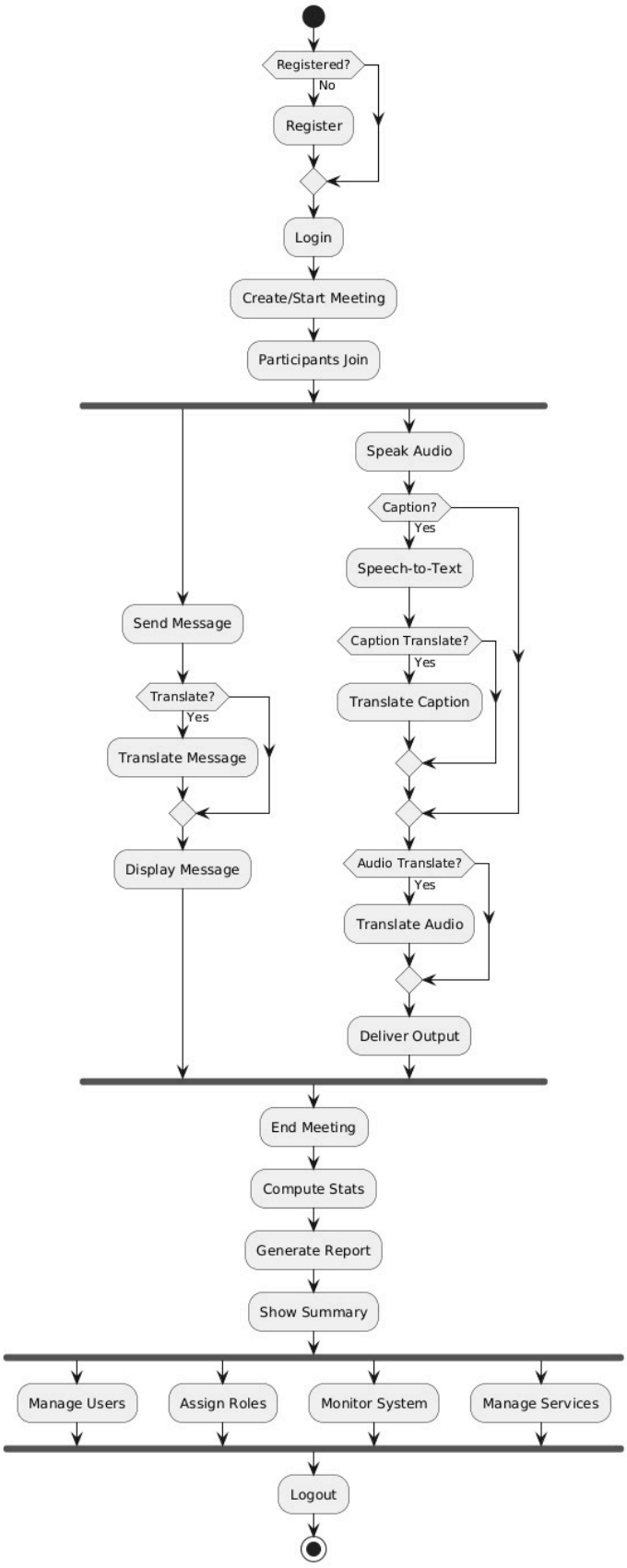
6. Report Service

- External service for generating reports.
- Used by Report Generator.
- Shows **service integration**.

KEY RELATIONSHIPS

- Client → Web Server → Application Server (request flow)
- Application Server → Database (data storage)
- Application Server → External Services (Translation API, Report Service)
- Admins → Admin UI → Backend services

ACITIVITY DIAGRAM :→


Registered?
No
Register
Login
Create/Start Meeting
Participants Join
Speak Audio
Caption?
Yes
Speech-to-Text
Caption Translate?
Yes
Translate Caption
Send Message
Translate?
Yes
Translate Message
Display Message
Audio Translate?
Yes
Translate Audio
Deliver Output
End Meeting
Compute Stats
Generate Report
Show Summary
Manage Users
Assign Roles
Monitor System
Manage Services
Logout

- This diagram shows the **workflow of the system**, representing the sequence of activities from start to end.

---

1. User Authentication Flow

- Process starts with checking **Registered?**
- If not registered → user performs **Register**
- Then user proceeds to **Login**
- This shows a **decision-based flow**

---

2. Meeting Initialization

- After login:
  - User **creates/starts meeting**
  - Participants **join the meeting**
- This represents the **start of interaction phase**

---

3. Parallel Activities (Chat & Audio)

- The diagram splits into two parallel flows:

Chat Flow

- User sends message
- System checks **Translate?**
- If yes → message is translated
- Then message is displayed

Audio Flow

- User speaks audio
- System checks **Caption?**
- If yes → speech is converted to text

- Then optional caption translation
- Then optional audio translation
- Finally output is delivered

👉 Shows **parallel processing + decision points**

---

4. Meeting Completion

- After interaction:
  - Meeting ends
  - System computes statistics
  - Generates report
  - Shows summary

---

5. Admin Activities

- After meeting:
  - Manage Users
  - Assign Roles
  - Monitor System
  - Manage Services
- Shows **administrative control flow**

---

6. End of Process

- User performs **Logout**
- Process ends

KEY RELATIONSHIPS

- Sequential flow (step-by-step execution)

- Decision nodes (yes/no conditions)
- Parallel flows (chat and audio together)
- Merge points (flows combine back)

Summary of SRS – Real-Time Multilingual Video Conferencing System

This Software Requirements Specification (SRS) describes a web-based system designed for real-time video communication with multilingual support. The main purpose of the system is to allow users who speak different languages to communicate easily using AI-based speech recognition and translation.

The system focuses on one-to-one video calls in its initial version. It uses technologies like WebRTC for real-time communication and integrates AI services for converting speech to text and translating it into the user's preferred language. Along with video and audio communication, it also provides live captions and translated chat messages, improving accessibility and usability.

Key features include:

Secure user authentication

Real-time video and audio calls

Live speech-to-text captions

Instant language translation during calls

Encrypted communication for privacy

Text chat with multilingual translation

The system is designed as a browser-based application, meaning users do not need to install additional software. It depends on a stable internet connection, modern browsers, and access to devices like a camera and microphone.

From a performance perspective, the system aims to maintain low latency (under 300 ms) and proper audio-video synchronization. Security is a major focus, ensuring encrypted communication and preventing unauthorized access.

The SRS also defines system usability and reliability, emphasizing a simple interface and stable performance even under minor network issues.

Overall, this system aims to break language barriers in communication, making it useful for professionals such as doctors, lawyers, educators, and organizations requiring secure multilingual interaction.